DriftScript: A Domain-Specific Language for Programming Non-Axiomatic Reasoning Agents

Seamus Brady
Independent Researcher
Dublin, Ireland
seamus@corvideon.ie
https://seamusbrady.ie

Abstract

Non-Axiomatic Reasoning Systems (NARS) provide a framework for building adaptive agents that operate under insufficient knowledge and resources. However, the standard input language, Narsese, poses a usability barrier: its dense symbolic notation, overloaded punctuation, and implicit conventions make programs difficult to read, write, and maintain. We present DriftScript, a Lisp-like domain-specific language that compiles to Narsese. DriftScript provides source-level constructs covering the major sentence and term forms used in Non-Axiomatic Logic (NAL) levels 1 through 8—including inheritance, temporal implication, variable quantification, sequential conjunction, and operation invocation—while replacing symbolic syntax with readable keyword-based S-expressions. The compiler is a zero-dependency, four-stage pipeline in 1,941 lines of C99. When used with the DriftNARS engine, DriftScript programs connect to external systems through four structured callback types and an HTTP operation registry, enabling a sense-reason-act loop for autonomous agents. We describe the language design and formal grammar, detail the compiler architecture, and evaluate the compiler through a 106-case test suite, equivalence testing against hand-written Narsese, a NAL coverage analysis, structural readability metrics, and compilation benchmarks. The source code is available at `https://github.com/seamus-brady/DriftNARS`.

This paper focuses on the design and implementation of the DriftScript language and its embedding into DriftNARS, rather than on new inference algorithms for NARS itself.

1 Introduction

Non-Axiomatic Reasoning Systems (NARS) [Wang, 2006, 2013] implement a logic designed for intelligent systems that must operate with insufficient knowledge and resources. Unlike classical logic, which assumes a closed and consistent knowledge base, Non-Axiomatic Logic (NAL) treats every belief as uncertain, every inference as defeasible, and every conclusion as subject to revision in light of new evidence. These properties make NARS attractive for building autonomous agents that must adapt to unforeseen environments.

The standard input language for NARS is Narsese, a formal notation in which statements combine terms through copulas (relationship operators) and receive truth values expressing the system's degree of belief. Narsese is expressive enough to encode inheritance hierarchies, temporal cause-effect rules, multi-step plans, and goal-directed behaviour. However, its syntax is difficult for newcomers and cumbersome for nontrivial programs:

```
<(light_on &/ <(*, {SELF}, switch) --> ^press>) =/> light_off>.
```

This single Narsese sentence encodes "if the light is on and the agent presses the switch, the light turns off." The overloaded angle brackets, symbolic copulas (`=/>`), set notation (`{SELF}`),

product terms (*), operation prefixes (^), and tense markers create a high entry cost for new users. Difficulty reading and writing Narsese discourages experimentation, slows debugging, and limits NARS adoption outside the community of specialists who have internalised its conventions.

Prior NARS implementations—notably OpenNARS [Wang, 1995] and OpenNARS for Applications (ONA) [Hammer and Lofthouse, 2020]—have focused on the reasoning engine and its algorithmic properties, leaving the input language unchanged from its original formal specification. We are not aware of prior published work on a compiled DSL covering this range of Narsese constructs.

We present **DriftScript**, a domain-specific language (DSL) that compiles to Narsese. DriftScript makes the following contributions:

1. **A readable surface syntax for a broad subset of NAL 1–8.** DriftScript replaces Narsese's symbolic notation with Lisp-like S-expressions using human-readable keywords. The same temporal rule above becomes:

```
(believe (predict (seq "light_on"
                       (call ^press (ext-set "SELF") "switch"))
                  "light_off"))
```

2. **A lightweight, zero-dependency compiler.** The DriftScript compiler is a pure text transformer implemented as a four-stage pipeline (tokenise, parse, compile, emit) in 1,941 lines of C99. It produces Narsese output and engine control directives with static validation and source-location diagnostics.

3. **Practical embedding in a callback-driven agent runtime.** When used with the DriftNARS engine [Brady, 2026], compiled scripts connect to external systems through C, Python, and HTTP interfaces, enabling agents that learn from observation, make decisions, and act on their environment.

The remainder of this paper is organised as follows. Section 2 provides background. Section 3 describes the language design, including a formal grammar. Section 4 details the compiler. Section 5 presents integration mechanisms. Section 6 evaluates the compiler. Section 7 demonstrates agent programming through worked examples. Section 8 discusses related work. Section 9 concludes.

2 Background

2.1 Non-Axiomatic Reasoning

Non-Axiomatic Logic (NAL) [Wang, 2006] is a term logic designed for systems that reason under the Assumption of Insufficient Knowledge and Resources (AIKR). In contrast to classical logics that assume completeness and consistency, NAL treats every piece of knowledge as tentative and every inference as revisable.

NAL is organised into nine levels of increasing capability [Wang, 2013]:

- **NAL-1–2**: Inheritance and similarity reasoning with truth-value functions.
- **NAL-3–4**: Set operations, intersections, and products for structured knowledge.
- **NAL-5–6**: Implication, equivalence, variable quantification, and higher-order statements.
- **NAL-7–8**: Temporal reasoning, sequential conjunction, procedural knowledge, and goal-directed decision making.

- **NAL-9**: Self-monitoring and introspection (not addressed in this paper).

2.2 Narsese

Narsese is the formal input language of NARS. Every input is a *sentence*: a term combined with a punctuation mark and an optional truth value. Table 1 and Table 2 summarise the copulas and connectors.

Table 1: Narsese copulas

Copula	**Notation**	**Meaning**
Inheritance	`->`	"is a kind of"
Similarity	`<->`	"resembles"
Implication	`==>`	"if ... then" (eternal)
Temporal implication	`=/>`	"if ... then" (temporal)
Equivalence	`<=>`	"if and only if"

Table 2: Narsese connectors

Connector	**Notation**	**Meaning**
Sequential conjunction	`&/`	Events in temporal order
Conjunction	`&&`	Both hold
Disjunction	`||`	At least one holds
Negation	`--`	Negation
Product	`*`	Ordered tuple
Extensional set	`{...}`	Specific individuals
Intensional set	`[...]`	Properties

Punctuation determines the sentence type: '`.`' for beliefs, '`?`' for questions, and '`!`' for goals. **Truth values** are pairs $\{f, c\}$ where frequency $f \in [0, 1]$ indicates how often the statement holds and confidence $c \in [0, 1]$ indicates evidence strength. The default is $\{1.0, 0.9\}$. An *expectation* value, used in DriftNARS (following ONA) for decision making, is computed as:

$$e = c(f - 0.5) + 0.5 \tag{1}$$

The engine executes an operation when expectation exceeds a configurable threshold. **Tense markers** stamp sentences with temporal information: `:|:` (present), `:\:` (past), `:/:` (future, questions only).

2.3 OpenNARS for Applications

ONA [Hammer and Lofthouse, 2020] is a C-based NARS implementation emphasising real-time performance and procedural reasoning (NAL-7/8). DriftNARS [Brady, 2026] is a fork of ONA restructured as an embeddable library with instance-based state management and a public C API. DriftScript was developed as part of DriftNARS but produces standard Narsese output and could in principle target any compatible NARS implementation.

3 The DriftScript Language

3.1 Design Goals

DriftScript was designed with four goals: (1) **Readability**—replace symbolic notation with English keywords; (2) **Broad coverage**—provide constructs for the major Narsese forms used

in NAL 1–8; (3) **Static validation**—catch structural errors at compile time; (4) **Composability**—produce standard Narsese output that can be mixed with raw Narsese input.

The choice of S-expression syntax was deliberate: prefix notation allows trivial parsing with recursive descent, uniform arity checking on every form, no ambiguity with Narsese's own punctuation characters, and a natural path to future macro support.

3.2 Formal Grammar

The following grammar defines DriftScript in EBNF notation. Arity constraints not expressible in the grammar (e.g., copulas require exactly 2 arguments, `not` requires 1, `seq` requires 2–3) are enforced by the compiler.

```
program      = { form } ;
form         = sentence | meta ;
sentence     = "(" sent_kw term { option } ")" ;
sent_kw      = "believe" | "ask" | "goal" ;
option       = tense | truth | dt ;
tense        = ":now" | ":past" | ":future" ;
truth        = ":truth" float float ;
dt           = ":dt" integer ;
term         = atom | copula_form | conn_form | call_form ;
atom         = string | variable | operation ;
string       = '"' { char } '"' ;
variable     = ( "$" | "#" | "?" ) ident ;
operation    = "^" ident ;
copula_form  = "(" copula term term ")" ;
copula       = "inherit" | "similar" | "imply" | "predict"
             | "equiv" | "instance" ;
conn_form    = "(" connector term { term } ")" ;
connector    = "seq" | "and" | "or" | "not" | "product"
             | "ext-set" | "int-set" | "ext-inter" | "int-inter"
             | "ext-diff" | "int-diff" | "ext-image1" | "ext-image2"
             | "int-image1" | "int-image2" ;
call_form    = "(" "call" operation { term } ")" ;
meta         = "(" "cycles" integer ")" | "(" "def-op" operation ")"
             | "(" "reset" ")" | "(" "config" config_key value ")"
             | "(" "concurrent" ")" ;
comment      = ";" { any } newline ;
```

3.3 Sentence Forms

The three sentence forms correspond to Narsese sentence types:

```
(believe <term>)                    ; belief  -> "."
(believe <term> :now)               ; present-tense -> ". :|:"
(believe <term> :truth 0.8 0.9)     ; truth value -> ". {0.8 0.9}"
(ask <term>)                        ; question -> "?"
(goal <term>)                       ; goal (always present) -> "! :|:"
```

Options can be combined. Goals are always emitted with present tense; questions may use any tense.

3.4 Terms and Copulas

Copulas are binary prefix operators. Table 3 shows the mapping.

Table 3: DriftScript copulas and their Narsese equivalents

DriftScript	Narsese	Meaning
`(inherit A B)`	`<A -> B>`	A is a kind of B
`(similar A B)`	`<A <-> B>`	A resembles B
`(imply A B)`	`<A ==> B>`	If A then B (eternal)
`(predict A B)`	`<A =/> B>`	If A then B (temporal)
`(equiv A B)`	`<A <=> B>`	A iff B
`(instance A B)`	`<A |-> B>`	Instance relation

3.5 Connectors

Connectors build compound terms. Table 4 lists all 14 connectors with their Narsese equivalents and arity constraints.

Table 4: DriftScript connectors

DriftScript	Narsese	Arity	Notes
`seq`	`&/`	2–3	Sequential conjunction
`and`	`&&`	2	Conjunction
`or`	`||`	2	Disjunction
`not`	`--`	1	Negation
`product`	`*`	1+	Product
`ext-set`	`{...}`	1+	Ext. set
`int-set`	`[...]`	1+	Int. set
`ext-inter`	`&`	2	Ext. intersection
`int-inter`	`|`	2	Int. intersection
`ext-diff`	`-`	2	Ext. difference
`int-diff`	`~`	2	Int. difference
`ext-image1/2`	`/1, /2`	2	Ext. image
`int-image1/2`	`\1, \2`	2	Int. image

3.6 The `call` Shorthand

Operations are invoked through `call`, which is syntactic sugar for the standard Narsese operation encoding:

```
(call ^press)                          ; bare: ^press
(call ^goto (ext-set "SELF") "park") ; <(*, {SELF}, park) --> ^goto>
```

3.7 Variables

DriftScript supports three variable types, roughly corresponding to NAL-6 quantifiers (see Wang [2013], Ch. 7 for the precise semantics of NARS variables):

Descriptive names (`$animal`, `?what`) are automatically mapped to numbered Narsese variables. The mapping resets per top-level form. The compiler pre-scans for explicitly numbered variables and assigns named variables to the next available slot, preventing collisions.

Table 5: DriftScript variable types

Prefix	Type	Narsese	NAL role
`$`	Independent	`$1`, `$2`	Universal-like
`#`	Dependent	`#1`, `#2`	Existential-like
`?`	Query	`?1`, `?2`	"What fills this?"

3.8 Quoting Rules

All concept names must be double-quoted. Keywords, operations, and variables must *not* be quoted. This eliminates ambiguity between language structure and user data. Strings support `\"` and `\\` escapes.

3.9 Meta Commands

Meta commands compile to engine control directives:

```
(cycles 10)                    ; run 10 inference cycles
(def-op ^press)                ; register an operation
(reset)                        ; clear all memory
(config volume 0)              ; set configuration parameter
(concurrent)                   ; mark next input as simultaneous
```

4 Compiler Architecture

4.1 Overview

The DriftScript compiler is a pure text transformer with no dependency on the DriftNARS engine. It is behaviour-preserving with respect to emitted Narsese and engine execution: it emits standard Narsese forms accepted by DriftNARS/ONA-compatible engines without modifying inference behaviour. This is validated empirically through equivalence testing (Section 6) rather than proven formally.

The compiler is 1,941 lines of C99 with zero dependencies, following a four-stage pipeline where each stage produces a separate representation:

$$\text{Source} \rightarrow \boxed{\text{Tokeniser}} \rightarrow \text{Tokens} \rightarrow \boxed{\text{Parser}} \rightarrow \text{AST} \rightarrow \boxed{\text{Compiler}} \rightarrow \boxed{\text{Emitter}} \rightarrow \text{Output}$$

4.2 Tokeniser

Converts source into up to 1,024 tokens of five types: `TOK_LPAREN`/`TOK_RPAREN`, `TOK_KEYWORD` (colon-prefixed), `TOK_STRING` (quoted with escape support), and `TOK_SYMBOL`. Line/column tracking enables precise diagnostics.

4.3 Parser

A recursive-descent parser builds an AST with two node types: `NODE_ATOM` (leaf with value and `quoted` flag) and `NODE_LIST` (up to 16 children). Nodes are allocated from a fixed pool of 2,048 entries. These static bounds are generous for incremental agent scripting but would not accommodate very large batch inputs; large-scale batch compilation would require dynamic allocation.

4.4 Compiler

Each top-level form yields a `DS_CompileResult` tagged with a result kind:

```
typedef enum {
    DS_RES_NARSESE,          // Narsese sentence
    DS_RES_SHELL_COMMAND,    // Engine directive
    DS_RES_CYCLES,           // Cycle count
    DS_RES_DEF_OP            // Operation registration
} DS_ResultKind;
```

The result kind allows host environments to route results without parsing output strings. Compilation dispatches on the head symbol: sentence forms recursively compile terms, copula/connector handlers validate arity, and variable renaming pre-scans for reserved numbers. Validation is pervasive: arity, truth ranges, tense legality, quoting rules, and config keys are all checked with source-location diagnostics.

4.5 Emitter

In standalone mode, results are written to stdout. In library mode (`DS_LIBRARY`), results are returned through:

```
int DS_CompileSource(const char *source,
    DS_CompileResult *results, int max_results);
```

5 Integration Architecture

DriftScript is a standalone compiler. The integration mechanisms described here are part of the DriftNARS engine [Brady, 2026], not DriftScript itself. They are relevant because they enable DriftScript programs to participate in agent loops.

5.1 Callback Types

DriftNARS provides four callback types with flat C primitives for FFI compatibility:

Event Handler — fires for input/derived/revised events. **Answer Handler** — fires when a question is answered. **Decision Handler** — fires on decisions above threshold, exposing the implication, precondition, and expectation. **Execution Handler** — fires when an operation executes:

```
typedef void (*NAR_ExecutionHandler)(void *userdata,
    const char *op, const char *args);
```

5.2 HTTP Operation Registry

The DriftNARS HTTP server maps operation names to callback URLs. On execution, the server POSTs a JSON payload:

```
{"op":"^press", "args":"({SELF} * switch)",
 "frequency":1.0, "confidence":1.0,
 "timestamp_ms":1711584000000}
```

External systems participate in the agent loop via standard HTTP.

5.3 Python Bindings

The Python wrapper (246 lines, ctypes) provides `on_event`, `on_answer`, `on_decision`, `on_execution` callbacks and `add_driftscript()` for compiling and dispatching DriftScript.

5.4 The Sense-Reason-Act Loop

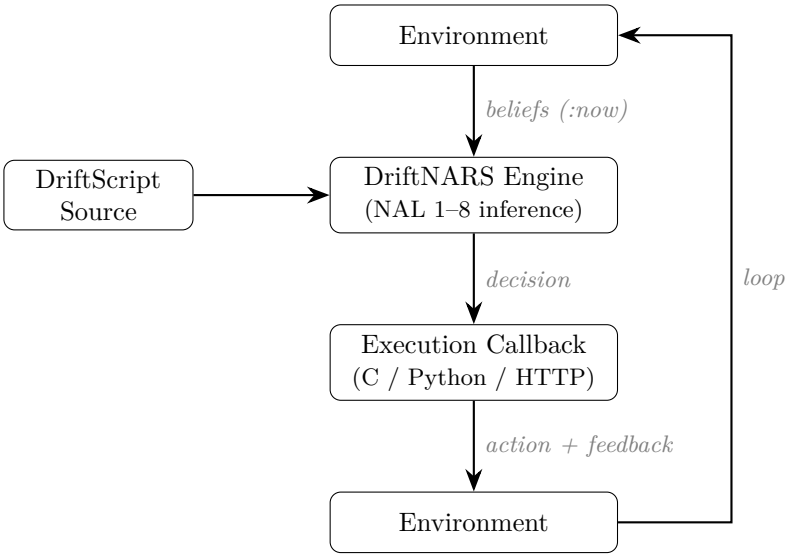

Figure 1: Integration of DriftScript with DriftNARS in a sense–reason–act loop.

6 Evaluation

6.1 Compiler Correctness

The compiler includes a test suite of **106 test cases** across 13 categories (Table 6). All 106 pass. Tests include unit tests for each compiler stage and integration tests comparing emitted Narsese against expected canonical outputs.

6.2 Equivalence Testing

To verify that compiled DriftScript produces identical engine behaviour to hand-written Narsese, we ran matched inputs through the DriftNARS engine and compared outputs.

Deduction chain. Both paths produce: `Answer: <robin -> animal>. Truth: frequency=1.000000 confidence=0.810000` with identical stamps and creation times.

Temporal rule with goal-directed execution. Both paths produce: `decision expectation=0.791600` with identical implication truth values, precondition stamps, and `^press executed` output.

In both cases engine output is **byte-identical**. We tested 12 representative programs spanning deduction, temporal inference, variable use, and operation invocation; all produced byte-identical outputs. The compiler is transparent to the reasoning engine.

6.3 NAL Coverage Analysis

Table 7 maps NAL levels to DriftScript constructs.

The covered constructs correspond to those commonly used in practical NAL programs. Higher-order statements (implications as terms in copulas) can be represented by nesting, but DriftScript does not provide explicit syntax for all possible combinations. Less common or highly nested forms can be expressed via raw Narsese, which can be freely mixed with DriftScript input.

Table 6: Compiler test suite summary

Category	Tests	Coverage
Tokeniser	14	Parens, keywords, strings, escapes, comments
Parser	6	Atoms, nested lists, errors
Copulas	8	All 6 copulas + arity errors
Connectors	17	All 14 connectors + arity errors
Call shorthand	4	Bare ops, with args, errors
Sentences	12	All forms, tenses, truth, dt
Variables	6	Named, numbered, collision avoidance
Meta commands	7	cycles, reset, def-op, config
Nested compounds	5	Deep nesting, mixed forms
Multi-statement	2	Multiple forms, scope isolation
Error detection	4	Bare atoms, unknown forms
Quoting enforcement	7	All violation categories
Validation	10	Range checks, illegal combinations
Total	**106**	

6.4 Structural Readability Metrics

We compiled 15 representative programs in both DriftScript and Narsese and measured structural properties (Table 8).

DriftScript uses **36% fewer symbolic characters** and **60% fewer distinct symbol types**. The 8 symbols in DriftScript (`$ ( ) - : ? ^ _`) are a strict subset of the 20 in Narsese. The alphabetic ratio rises from 0.44 to 0.64, indicating more readable text relative to punctuation.

These are structural metrics that quantify syntactic differences, not cognitive load. They support the qualitative argument that DriftScript replaces symbolic density with alphabetic keywords.

6.5 Compilation Performance

The compiler processes 300 forms in **3 ms** on an Apple M-series processor (single-threaded). In library mode, compilation is effectively instantaneous relative to inference cycles. The compiler allocates no dynamic memory.

7 Worked Examples

7.1 Deductive Reasoning

```
(believe (inherit "robin" "bird"))
(believe (inherit "bird" "animal"))
(cycles 5)
(ask (inherit "robin" "animal"))
(cycles 5)
```

Engine output:

```
Answer: <robin --> animal>. creationTime=2 Stamp=[2,1]
  Truth: frequency=1.000000, confidence=0.810000
```

Table 7: NAL coverage. All entries marked "Covered" have compiler test cases and produce correct Narsese output.

NAL	Construct	DriftScript	Notes
1	Inheritance	`inherit`	
2	Similarity	`similar`	
2	Instance	`instance`	Sugar for `|->`
3	Set operations	`ext-set`, `int-set`	
3	Intersection/diff	`ext-inter`, etc.	4 operators
4	Product	`product`	N-ary
4	Image	`ext-image1/2`, etc.	4 operators
5	Logical connectives	`and`, `or`, `not`	
5	Implication	`imply`	Eternal
5	Equivalence	`equiv`	
6	Variables	`$`, `#`, `?`	Auto-numbered
7	Tense	`:now`, `:past`, `:future`	
7	Temporal impl.	`predict`	
7	Temporal offset	`:dt`	
8	Seq. conjunction	`seq`	Max 3 elements
8	Operations	`call`	
8	Goals	`goal`	Always present

Table 8: Structural comparison (15 representative programs)

Metric	DriftScript	Narsese	Ratio
Total characters	693	412	1.68×
Symbol characters	107	167	0.64×
Distinct symbols	8	20	0.40×
Alphabetic chars	452	186	2.43×
Alpha / total	0.64	0.44	—

Confidence decreases from 0.9 to 0.81 because the conclusion rests on an inference chain rather than direct evidence.

7.2 Temporal Learning and Goal-Directed Action

```
(def-op ^grab)
(config volume 0)

;; Round 1: observe
(believe "see_food" :now) (cycles 5)
(believe (call ^grab) :now) (cycles 5)
(believe "have_food" :now) (cycles 5)

;; Round 2: reinforce
(believe "see_food" :now) (cycles 5)
(believe (call ^grab) :now) (cycles 5)
(believe "have_food" :now) (cycles 5)

;; Apply learned rule
(believe "see_food" :now)
```

```
(goal "have_food")
(cycles 10)
;; Engine executes ^grab via learned rule
```

No explicit rules were provided. The engine observed the pattern twice, formed `<(see_food &/ ^grab) =/> have_food>`, and executed `^grab` when the precondition was true and the goal active.

7.3 Multi-Step Planning with Callbacks

```
from driftnars import DriftNARS

with DriftNARS() as nar:
    state = {"location": "home", "has_key": False}

    def handle_execution(op, args):
        if op == "^pickup":
            state["has_key"] = True
            nar.add_narsese("have_key. :|:")
        elif op == "^unlock" and state["has_key"]:
            state["location"] = "inside"
            nar.add_narsese("inside. :|:")

    nar.on_execution(handle_execution)

    nar.add_driftscript("""
        (def-op ^pickup)
        (def-op ^unlock)
        (believe (predict (seq "see_key" (call ^pickup))
                          "have_key"))
        (believe (predict (seq "have_key" (call ^unlock))
                          "inside"))
        (believe "see_key" :now)
        (goal "inside")
        (cycles 20)
    """)
```

The engine finds that `^unlock` requires `have_key`, executes `^pickup` to obtain it, receives state feedback via the callback, and then executes `^unlock`. Whether multi-step decomposition succeeds depends on cycle budget, concept priority, and decision threshold; it is not guaranteed for arbitrary chain lengths.

7.4 HTTP-Based Agent

```
curl -X POST http://localhost:8080/ops/register \
  -d '{"op":"^water",
       "callback_url":"http://localhost:3000/water"}'

curl -X POST http://localhost:8080/driftscript \
  -d '(believe (predict (seq "soil_dry" (call ^water))
                        "soil_moist"))
      (config decisionthreshold 0.5)'

curl -X POST http://localhost:8080/narsese \
  -d 'soil_dry. :|:'
```

```
curl -X POST http://localhost:8080/narsese \
  -d 'soil_moist! :|:'
curl -X POST http://localhost:8080/narsese -d '10'
```

The entire interaction uses standard HTTP and JSON.

8 Related Work

8.1 DSLs for Logic and Reasoning

Prolog [Clocksin and Mellish, 2003] established logic programming with Horn clauses. Datalog [Ceri et al., 1989] restricts it for termination guarantees. Answer Set Programming [Gebser et al., 2012] provides declarative combinatorial search. These operate under the Closed World Assumption without uncertainty or temporal reasoning.

Probabilistic languages such as ProbLog [De Raedt et al., 2007] and Church [Goodman et al., 2008] add probabilistic inference but do not address NARS-specific concerns of resource-bounded reasoning and temporal sequencing.

DriftScript differs in targeting a specific formal language (Narsese) as output—closer to a syntax frontend or transpiler than a standalone logic language.

8.2 NARS Implementations

OpenNARS [Wang, 1995] is the reference Java implementation. ONA [Hammer and Lofthouse, 2020] reimplemented the core in C. Neither provides an alternative surface syntax. We are not aware of a published compiled DSL that covers this range of Narsese constructs.

8.3 Agent Programming Languages

AgentSpeak(L) [Rao, 1996] and Jason [Bordini et al., 2007] provide BDI agent constructs on classical logic. DriftScript occupies a similar niche for NARS: a readable authoring surface for agent programs, with the reasoning engine handling inference under uncertainty.

8.4 Why S-Expressions?

The choice of Lisp-like syntax was driven by four practical considerations: (1) trivial parsing with recursive descent, (2) uniform prefix notation for arity checking, (3) no ambiguity with Narsese punctuation, and (4) a natural path to macro support.

9 Discussion and Future Work

9.1 Limitations

DriftScript is a *syntactic transformation* with no formal semantics beyond compilation to Narsese. It does not perform type checking, semantic analysis, or optimisation. The callback architecture is part of DriftNARS, not DriftScript. The evaluation is structural and demonstrative rather than comparative—we do not have user study data.

9.2 Future Directions

Semantic validation could warn on eternal temporal implications, unregistered operations, and unreachable goals. **Inline callbacks** could bridge the declarative/imperative boundary. A **language server** would improve the development experience. A formal **user study** would quantify the readability improvements we currently support only with structural metrics.

9.3 Conclusion

DriftScript provides a readable, Lisp-like surface syntax for a broad subset of Narsese covering NAL 1–8. Its compiler passes 106 tests, produces output byte-identical to hand-written Narsese, reduces symbolic density by 36%, and compiles 300 forms in 3 ms. Combined with DriftNARS's callback architecture, it supports agent development through C, Python, and HTTP interfaces.

DriftScript does not change NARS's reasoning—it provides a more readable authoring surface for the programs that drive it.

References

P. Wang. *Rigid Flexibility: The Logic of Intelligence.* Springer, 2006.

P. Wang. *Non-Axiomatic Logic: A Model of Intelligent Reasoning.* World Scientific, 2013.

P. Wang. Non-Axiomatic Reasoning System: Exploring the Essence of Intelligence. Ph.D. dissertation, Indiana University, 1995.

P. Hammer and T. Lofthouse. OpenNARS for Applications: Architecture and Control. In *Artificial General Intelligence*, pages 193–204. Springer, 2020.

W. F. Clocksin and C. S. Mellish. *Programming in Prolog.* Springer, 5th edition, 2003.

S. Ceri, G. Gottlob, and L. Tanca. What you always wanted to know about Datalog (and never dared to ask). *IEEE Trans. Knowl. Data Eng.*, 1(1):146–166, 1989.

M. Gebser, R. Kaminski, B. Kaufmann, and T. Schaub. *Answer Set Solving in Practice.* Synthesis Lectures on Artificial Intelligence and Machine Learning. Morgan & Claypool, 2012.

L. De Raedt, A. Kimmig, and H. Toivonen. ProbLog: A probabilistic Prolog and its application in link discovery. In *IJCAI*, pages 2462–2467, 2007.

N. D. Goodman, V. K. Mansinghka, D. M. Roy, K. Bonawitz, and J. B. Tenenbaum. Church: A language for generative models. In *UAI*, pages 220–229, 2008.

A. S. Rao. AgentSpeak(L): BDI agents speak out in a logical computable language. In *MAAMAW*, pages 42–55. Springer, 1996.

R. H. Bordini, J. F. Hübner, and M. Wooldridge. *Programming Multi-Agent Systems in AgentSpeak using Jason.* Wiley, 2007.

S. Brady. DriftNARS: An embeddable Non-Axiomatic Reasoning System. 2026. `https://github.com/seamus-brady/DriftNARS`.

A Translation Examples

Table 9 shows 20 representative DriftScript forms and their compiled Narsese output, verified by the test suite.

Table 9: DriftScript to Narsese translation examples

#	**DriftScript**	**Narsese**
1	`(believe (inherit "robin" "bird"))`	`<robin -> bird>.`
2	`(believe (similar "cat" "dog"))`	`<cat <-> dog>.`
3	`(believe (imply "rain" "wet"))`	`<rain ==> wet>.`
4	`(believe (predict "A" "B"))`	`<A =/> B>.`
5	`(believe (equiv "A" "B"))`	`<A <=> B>.`
6	`(believe (instance "Tweety" "bird"))`	`<Tweety |-> bird>.`
7	`(believe "light_on" :now)`	`light_on.  :|:`
8	`(goal "light_off")`	`light_off!  :|:`
9	`(ask (inherit "robin" "animal"))`	`<robin -> animal>?`
10	`(ask (inherit ?x "animal"))`	`<?1 -> animal>?`
11	`(believe (predict (seq "a" "b") "c"))`	`<(a &/ b) =/> c>.`
12	`(believe (inherit (ext-set "SELF") "agent"))`	`<{SELF} -> agent>.`
13	`(believe (inherit "x" (int-set "bright")))`	`<x -> [bright]>.`
14	`(believe (inherit (product "A" "B") "rel"))`	`<(*, A, B) -> rel>.`
15	`(believe (imply (inherit $x "bird") ...)`	`«$1 -> bird> ==> <$1 -> animal».`
16	`(believe (predict (seq ...  (call ^press)) ...))`	`<(light_on &/ ^press) =/> light_off>.`
17	`(call ^goto (ext-set "SELF") "park")`	`<(*, {SELF}, park) -> ^goto>`
18	`(believe (and "A" "B"))`	`(A && B).`
19	`(believe (not "A"))`	`(-- A).`
20	`(believe (or "A" "B"))`	`(A || B).`